

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 - Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	S.MAHALAKSHMI	Reg. No.:	192524372
Section / Batch:	2 ND YEAR	Date:	31/08/26

Instructions to Students

- Answer ALL questions with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions	Marks
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – Pseudocode Writing Task

CO5_AT2_PSEUDOCODE WRITING TASK

Q1. GRAPH COLORING PROBLEM WITH M COLORS

Course: Design and Analysis of Algorithms (DAA)

Problem Statement

Develop a pseudocode using backtracking to color the vertices of a graph using at most M colors such that adjacent vertices have different colors. The algorithm should assign colors vertex by vertex while checking adjacency constraints. If a valid coloring is not possible, the algorithm should backtrack and try another color.

Algorithmic Approach

The problem is solved using the Backtracking technique.

The vertices are colored one by one. For each vertex, the algorithm tries all colors from 1 to M. Before assigning a color, it checks whether the color is already assigned to any adjacent vertex. If the color is safe, it is assigned and the algorithm recursively processes the next vertex. If no color is possible, the algorithm backtracks to the previous vertex and tries another color.

Pseudocode

```
ALGORITHM M_COLORING(G, V, M)

INPUT:
    G → Graph with V vertices
    M → Maximum number of colors

OUTPUT:
    A valid coloring of all vertices if possible

BEGIN
    color[1...V] ← 0

    IF COLOR_GRAPH(1) = TRUE THEN
        PRINT "Valid Coloring"

        FOR i ← 1 TO V DO
            PRINT "Vertex", i, "→ Color", color[i]
        END FOR
    ELSE
        PRINT "No valid coloring possible with M colors"
    END IF
END
```

```

FUNCTION COLOR_GRAPH(vertex)

BEGIN
    IF vertex > V THEN
        RETURN TRUE
    END IF

    FOR c ← 1 TO M DO
        IF IS_SAFE(vertex, c) = TRUE THEN

            color[vertex] ← c

            IF COLOR_GRAPH(vertex + 1) = TRUE THEN
                RETURN TRUE
            END IF

            color[vertex] ← 0
        END IF
    END FOR

    RETURN FALSE
END

FUNCTION IS_SAFE(vertex, c)

BEGIN
    FOR each adjacent vertex u of vertex DO
        IF color[u] = c THEN
            RETURN FALSE
        END IF
    END FOR

    RETURN TRUE
END

```

Logical Explanation

The algorithm starts with the first vertex and assigns a color to it. It then moves to the next vertex and checks whether the selected color conflicts with any of its adjacent vertices.

If there is no conflict, the color is assigned and the algorithm proceeds recursively to the next vertex.

If a conflict occurs, that color is rejected and another color is tried.

If none of the available colors can be assigned to the current vertex, the algorithm performs backtracking. It removes the color assigned to the previous vertex and tries another possible color.

This process continues until all vertices are successfully colored or all possible color combinations have been explored.

State-Space Representation

The state-space tree represents all possible color assignments for the vertices.

For $M = 3$ colors:

- Level 1 → Color assigned to Vertex 1
- Level 2 → Color assigned to Vertex 2
- Level 3 → Color assigned to Vertex 3
- Level 4 → Color assigned to Vertex 4
- And so on.

Each node in the state-space tree represents a partial coloring of the graph. Invalid branches are immediately removed when an adjacent vertex has already been assigned the same color.

Recursion Steps

1. Start with Vertex 1.
2. Try each color from 1 to M .
3. Check whether the selected color is safe.
4. If the color is safe, assign it to the vertex.
5. Recursively call the function for the next vertex.
6. If all vertices are colored, return TRUE.
7. If no color is possible for a vertex, remove the previous color assignment.
8. Try another color for the previous vertex.
9. Continue until a valid coloring is found or all possibilities are exhausted.

Pruning Condition

A branch is pruned when the selected color is already assigned to an adjacent vertex.

```
IF color[u] = c
    RETURN FALSE
```

This prevents the algorithm from continuing with an invalid partial solution and reduces unnecessary searching.

Correctness

10. The algorithm considers every possible color for each vertex.
11. A color is assigned only when it does not conflict with any already-colored adjacent vertex.
12. If a particular color assignment cannot lead to a complete solution, the algorithm backtracks and tries another possibility.

13. Therefore, if a valid coloring using at most M colors exists, the algorithm will find it.
14. If all possible assignments are exhausted, no valid coloring exists with the given M colors.

Complexity Analysis

Let V = Number of vertices and M = Number of available colors.

For every vertex, there can be up to M possible color choices. Therefore, the maximum number of possible color combinations is M^V .

If checking whether a color is safe takes $O(V)$ time in the worst case, the overall worst-case time complexity is:

Time Complexity: $O(V \times M^V)$

The color array and recursion stack require:

Auxiliary Space Complexity: $O(V)$

If an adjacency matrix is used to represent the graph, the graph requires $O(V^2)$ space. Therefore, with an adjacency matrix, total space complexity is $O(V^2)$.

Complexity Class

The general M -Coloring Decision Problem is NP-Complete.

Conclusion

The Graph Coloring Problem with M Colors can be effectively solved using the backtracking technique. The algorithm assigns colors to vertices one by one, checks adjacency constraints, and backtracks whenever a valid color cannot be assigned. The pruning condition avoids exploring invalid branches. The worst-case time complexity is $O(V \times M^V)$, while the auxiliary space complexity is $O(V)$.

Q2. SUBSET SUM PROBLEM FOR LISTING ALL VALID SUBSETS

Course: Design and Analysis of Algorithms (DAA)

Problem Statement

Given a set of integers and a target sum, develop a pseudocode using backtracking to generate all subsets whose sum is equal to the given target value. The algorithm should systematically explore every possible subset using recursive branching. It should include pruning conditions to avoid unnecessary searches when the running sum exceeds the target value.

Algorithmic Approach

The problem is solved using the Backtracking technique.

The algorithm considers each element of the set and makes two choices: include the current element in the subset or exclude the current element. The algorithm recursively explores both possibilities. Whenever the current subset sum becomes equal to the target, the subset is recorded as a valid solution.

If the running sum exceeds the target, the branch is pruned because adding more positive elements cannot produce the required target.

Input and Output

Input:

- Set of integers: $A[1...N]$
- Target sum: T

Output:

List of all subsets whose sum is equal to T .

Pseudocode

```
ALGORITHM SUBSET_SUM(A, N, T)
```

```
INPUT:
```

```
    A → Set of N integers
```

```
    T → Target sum
```

```
OUTPUT:
```

```
    All subsets whose sum is equal to T
```

```

BEGIN
    subset ← empty list

    PRINT "Valid Subsets:"

    FIND_SUBSETS(1, 0)
END

FUNCTION FIND_SUBSETS(index, currentSum)

BEGIN
    IF currentSum = T THEN
        PRINT subset
        RETURN
    END IF

    IF index > N THEN
        RETURN
    END IF

    IF currentSum > T THEN
        RETURN
    END IF

    // Include the current element
    ADD A[index] TO subset

    FIND_SUBSETS(index + 1, currentSum + A[index])

    REMOVE A[index] FROM subset

    // Exclude the current element
    FIND_SUBSETS(index + 1, currentSum)
END

```

Logical Explanation

The algorithm starts with an empty subset and a current sum of zero.

For every element, the algorithm has two choices.

Choice 1: Include the Element

The current element is added to the subset and its value is added to the current sum. The algorithm then recursively processes the next element.

Choice 2: Exclude the Element

The current element is removed from the subset and the algorithm recursively processes the next element without changing the current sum.

When currentSum equals the target, the current subset is a valid solution and is printed.

Example

Consider $A = \{2, 3, 5, 7\}$ and Target = 10.

Some valid subsets are:

- $\{2, 3, 5\}$
- $\{3, 7\}$

Therefore, the valid subsets whose sum is 10 are $\{2, 3, 5\}$ and $\{3, 7\}$.

State-Space Representation

The state-space tree represents the inclusion and exclusion decisions for each element.

- At every level, the left branch represents including the element.
- At every level, the right branch represents excluding the element.
- Each node represents a partial subset and its current sum.
- Branches that cannot produce the target are pruned.

Recursion Steps

15. Start with index = 1 and currentSum = 0.
16. Consider the current element.
17. Include the element in the subset.
18. Recursively process the next element.
19. Remove the element from the subset.
20. Exclude the element.
21. Recursively process the next element.
22. If currentSum equals the target, print the subset.
23. If currentSum exceeds the target, prune that branch.
24. Continue until all possible subsets have been explored.

Pruning Condition

The main pruning condition is:

```
IF currentSum > T
    RETURN
```

This prevents the algorithm from exploring a branch when the running sum has already exceeded the target. This pruning condition is applicable when the input elements are non-negative integers. If negative numbers are allowed, this pruning condition cannot be used directly because adding a negative number could reduce the current sum.

Correctness

25. The algorithm considers both possibilities for every element: including it and excluding it.
26. Therefore, every possible subset is explored.
27. Whenever the current sum becomes equal to the target, the corresponding subset is printed as a valid solution.
28. The backtracking step removes an included element before exploring the exclusion branch, ensuring that the subset is correctly maintained.
29. Thus, the algorithm produces all valid subsets whose sum is equal to the target without duplication.

Complexity Analysis

For N elements, every element has two choices: include the element or exclude the element. Therefore, the maximum number of subsets is 2^N .

In the worst case, the algorithm may explore all possible subsets.

Time Complexity: $O(2^N)$

If printing or copying a subset takes $O(N)$ time, the total output-related cost can be up to $O(N \times 2^N)$.

The recursion depth can be at most N , and the subset requires $O(N)$ space.

Auxiliary Space Complexity: $O(N)$

Complexity Class

The Subset Sum Decision Problem is NP-Complete.

The problem of listing all valid subsets can require exponential time because there can be up to 2^N possible subsets.

Conclusion

The Subset Sum Problem can be solved using backtracking by making an include-or-exclude decision for every element. Whenever the current sum reaches the target, the subset is recorded as a valid solution. Branches whose sum exceeds the target are pruned when all elements are non-negative. The worst-case time complexity is $O(2^N)$, with $O(N)$ auxiliary space.

Code of Conduct

I S.MAHALAKSHMI(192524372)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.MAHALAKSHMI _____

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Question No.	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)	Total Marks
Q1 (40 Marks)						
Q2 (30 Marks)						
Q3 (30 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____